AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

Deep Learning Assignment 1

MicroGrad: Scalar Automatic Differentiation Engine

Student Name	Seif eldin tamer safwat
Student ID	23p0240
Program	Computer and Artificial Intelligence Engineering
Course	Deep Learning
Assignment	Assignment 1 — MicroGrad

Documentation Report

Abstract

This project implements a small scalar automatic differentiation engine inspired by MicroGrad. Each scalar is represented by a **Value** object that stores its numerical data, gradient, and references to the values that produced it. Arithmetic operations construct a computational graph during the forward pass. A reverse traversal of that graph applies local derivatives and the chain rule to calculate gradients. On top of this engine, the project implements a simple neural-network API consisting of neurons, layers, and a multilayer perceptron (MLP). The implementation was checked against the supplied test suite, with all 12 tests passing in the recorded local run.

1. Introduction and Objectives

The purpose of the assignment is to understand the mechanics behind automatic differentiation and backpropagation by implementing them directly rather than relying on a deep-learning framework. The work focuses on scalar computation, computational-graph construction, local derivatives, topological sorting, gradient accumulation, and a basic neural-network interface.

Main objectives

• Represent scalar values and track their gradients.

• Overload Python operators so mathematical expressions build a computational graph.

• Implement derivatives for addition, multiplication, powers, tanh, and exponential.

• Use topological sorting to ensure gradients propagate in the correct order.

• Build Neuron, Layer, and MLP classes using the scalar autograd engine.

• Validate the implementation with unit tests and a small training demonstration.

2. Project Structure

```
assignment-1-micrograd-template/
■■■ src/
■   ■■■ __init__.py
■   ■■■ engine.py       # Value class and automatic differentiation
■   ■■■ nn.py           # Neuron, Layer, and MLP
■■■ tests/
■   ■■■ test_engine.py  # Tests for scalar engine
■   ■■■ test_nn.py      # Tests for neural-network API
■■■ MicroGrad_Demo.ipynb
■■■ README.md
```

The **engine.py** module contains the computational graph and differentiation logic. The **nn.py** module uses Value objects to construct trainable neural-network components. The tests directory contains the provided automated tests. The demo notebook imports the project modules and demonstrates operations, gradients, graph visualization, and MLP training.

3. Computational Graph and Value Class

A Value object represents one scalar in the computation. It stores the scalar's numerical value and the information needed to differentiate an expression involving it.

Attribute	Purpose
data	Numerical scalar represented by the node.
grad	Accumulated derivative of the final output with respect to this node.
_prev	Set of parent nodes used to create the current node.
Attribute	**Purpose**
_op	Operation that produced the current node.
_backward	Callable that propagates the current node's gradient to its parents.
label	Optional human-readable label, useful for debugging and graph visualization.

The graph is created lazily as operations are evaluated. For example, if c = a × b, the new node c stores a and b as its parents. Its local backward function knows how to contribute to the gradients of a and b.

4. Mathematical Derivatives

4.1 Addition

For c = a + b, the local derivatives are:

$$\partial c/\partial a = 1,\ \partial c/\partial b = 1$$

If the gradient arriving at c is $\partial L/\partial c$, the chain rule gives $\partial L/\partial a \mathrel{+}= \partial L/\partial c$ and $\partial L/\partial b \mathrel{+}= \partial L/\partial c$.

4.2 Multiplication

For c = a × b, the product rule gives:

$$\partial c/\partial a = b,\ \partial c/\partial b = a$$

The backward function therefore adds b × c.grad to a.grad and a × c.grad to b.grad. The use of += is essential because a node can contribute to the output through multiple paths.

4.3 Power

For c = a■, where n is an integer or floating-point scalar:

$$\partial c/\partial a = n \times a■■^{1}$$

The implementation supports scalar exponents and explicitly rejects a Value object as the exponent, matching the assignment's stated constraint.

4.4 Negation, subtraction, and division

Negation is implemented as multiplication by −1. Subtraction is implemented as addition of the negated second operand: a − b = a + (−b). Division is implemented using the reciprocal: a / b = a × b■¹. Consequently, division uses the power operation's derivative. Division by zero raises Python's ZeroDivisionError during the forward calculation.

4.5 Hyperbolic tangent

For y = tanh(x), the derivative is:

$$dy/dx = 1 - \tanh^2(x)$$

The forward value is computed using math.tanh. During backpropagation, the incoming gradient is multiplied by 1 − y^2.

4.6 Exponential

For y = e■, the derivative is:

$$dy/dx = e■ = y$$

The implementation reuses the computed output value y during the backward pass.

5. Backpropagation and Topological Sorting

Backpropagation applies the chain rule from the output node toward the input nodes. A node must receive all gradient contributions from nodes that depend on it before its own backward function is executed. A topological ordering provides this execution order.

Topological sorting procedure

• Start from the output node with an empty ordered list and an empty visited set.

• Visit each parent recursively if it has not already been visited.

• After visiting all parents, append the current node to the ordered list (post-order DFS).

• Reverse the resulting list for backpropagation.

The visited set is important for shared subexpressions and diamond-shaped graphs. For example, in c = a + a, the same node a is referenced twice. Gradient contributions must accumulate rather than overwrite one another.

Backward pass

```
topo = output._get_topo()
output.grad = 1.0 for node
in reversed(topo):
node._backward()
```

The output gradient is initialized to 1 because the derivative of the output with respect to itself is 1. Each node's local backward function then applies its derivative and adds the resulting contribution to its parent gradients.

6. Neural Network API

6.1 Neuron

A neuron with n inputs has n trainable weights and one trainable bias. The weights and bias are initialized independently using random values sampled uniformly from [−1, 1]. For inputs x■, …, x■, the pre-activation is:

$$z = \Sigma_{■} w_{■} x_{■} + b$$

The neuron returns tanh(z). Its parameters method returns the weights and bias, while zero_grad resets their gradients.

6.2 Layer

A Layer(nin, nout) contains nout neurons, each receiving nin inputs. The layer evaluates all neurons on the same input vector. If it contains one neuron, it returns a single Value; otherwise, it returns a list of Values. Its parameters method flattens the parameters of all neurons.

6.3 Multilayer perceptron

An MLP is a sequence of fully connected layers. For MLP(nin, [n■, n■, …, n■]), the layer dimensions are nin → n■ → n■ → … → n■. The output of each layer is passed as the input to the next layer. The MLP exposes all trainable parameters and provides a method to reset their gradients.

6.4 Parameter-count example

For MLP(3, [4, 4, 1]), the first layer has 4 × (3 + 1) = 16 parameters, the second has 4 × (4 + 1) = 20, and the output layer has 1 × (4 + 1) = 5. The total is 16 + 20 + 5 = 41 parameters.

7. Demonstration and Training Experiment

The accompanying notebook demonstrates scalar operations, activation functions, backpropagation, graph visualization, and construction of a small MLP. It also trains an MLP on the four examples of the XOR truth table. The input examples are [0,0], [0,1], [1,0], and [1,1], with targets 0, 1, 1, and 0 respectively.

The network architecture used in the demonstration is MLP(2, [4, 4, 1]). The model is trained using mean squared error and gradient descent. At each training iteration, predictions and loss are calculated in the forward pass, loss.backward() computes gradients, and each parameter is updated by subtracting the learning rate multiplied by its gradient. Gradients are reset before the next iteration.

Observed predictions from the completed notebook run

Input	Target	Prediction
[0, 0]	0.0	0.0013
[0, 1]	1.0	0.9577
[1, 0]	1.0	0.9592
[1, 1]	0.0	0.0021

All four predictions are close to their target values in this recorded run. Since the output neuron uses tanh, its output is bounded between −1 and 1. The predictions are therefore approximate rather than required to equal the target values exactly. The notebook also plots the training loss across epochs to visualize optimization.

8. Testing and Validation

The implementation was tested using the repository's supplied pytest suite. The recorded command was `python -m pytest -v`. The run collected 12 tests and reported 12 passed in 0.12 seconds.

Test area	Purpose	Result
Value initialization	Checks stored data, initial gradient, label, and representation.	Passed
Basic operations	Checks arithmetic and reverse arithmetic.	Passed
Type errors	Checks rejection of strings and lists in arithmetic.	Passed
Power	Checks forward power, power gradient, and exponent constraint.	Passed
Activations and exp	Checks tanh and exponential forward/backward calculations.	Passed
Diamond graph	Checks gradient accumulation when a node is reused.	Passed
Chain rule	Checks gradients of a composite expression.	Passed
Division by zero	Checks ZeroDivisionError.	Passed
Type casting	Checks int() and float() conversions.	Passed
Neuron, Layer, MLP	Checks neural-network components.	Passed

Passing the local test suite confirms that the implementation satisfies the cases in the provided tests. It does not, by itself, guarantee that any separate remote autograder or submission workflow has completed successfully.

9. Design Decisions and Justifications

- **Scalar Value objects:** Keeping one scalar per node makes the computational graph and chain rule explicit and easy to inspect.
- **Operator overloading:** Python arithmetic syntax makes expressions concise while automatically recording graph dependencies.
- **Parent sets:** A set stores unique parent nodes and avoids revisiting the same node during graph traversal.
- **Backward closures:** Each operation stores its own local derivative rule, separating local calculus from global traversal.
- **Gradient accumulation:** Gradients are added because a node may influence the output through multiple paths.
- **Depth-first topological sorting:** Post-order DFS ensures parent nodes appear before dependent nodes in the forward topological order; reversing it gives the correct backward execution order.
- **Modular neural-network API:** Neuron, Layer, and MLP separate model construction into reusable components.

10. Limitations and Future Improvements

This is an educational scalar engine rather than a production deep-learning framework. It operates on scalar values, so larger tensor computations are not vectorized. The current training example is a small generated XOR dataset and is intended to demonstrate the training process, not to evaluate generalization on a real-world dataset. Potential extensions include additional operations and activation functions, numerical gradient checking, improved graph rendering, more systematic training utilities, and experiments on a separately loaded dataset.

11. Conclusion

The project demonstrates how automatic differentiation can be built from elementary mathematical rules. The Value class records a computational graph and implements local derivatives, while topological sorting organizes reverse-mode gradient propagation. The neural-network API uses these scalar operations to construct and train an MLP. The supplied test suite passed all 12 tests in the recorded run, and the notebook's XOR experiment produced predictions close to all four target values.